



Programmieren

Falk Morgenstern
Leon Johann Brettin

Foliensatz basierend auf Unterlagen von Prof. Huhn, Prof. Meyer und weiteren.
Nur zur Verwendung in der Vorlesung „Programmieren“ an der Ostfalia.
Keine Weitergabe.



Programmieren

9. Modularisierung



Programmieren

9. Modularisierung

9.1 Pakete



Idee

- **Paket** = Sammlung zusammengehöriger Klassen (Bibliothek), Kompilationseinheit
- Zweck
 - mehr **Ordnung** in Programme bringen
 - bessere Kontrolle der **Zugriffsrechte** (wer darf auf was zugreifen)
 - Vermeidung von **Namenskonflikten**
- Beispiele für Pakete in der Java-Klassenbibliothek (Auswahl; `java.lang` wird ohne import-Anweisung automatisch in jeden Code importiert)

Paket	Wichtige Klassen
<code>java.lang</code>	<code>System</code> , <code>String</code> , <code>Integer</code> , <code>Character</code> , <code>Object</code> , <code>Math</code> , ...
<code>java.io</code>	<code>File</code> , <code>InputStream</code> , <code>OutputStream</code> , <code>Reader</code> , <code>Writer</code> , ...
<code>java.awt</code>	<code>Button</code> , <code>CheckBox</code> , <code>Frame</code> , <code>Color</code> , <code>Cursor</code> , <code>Event</code> , ...
<code>java.util</code>	<code>ArrayList</code> , <code>Hashtable</code> , <code>BitSet</code> , <code>Stack</code> , <code>Vector</code> , <code>Random</code> , ...



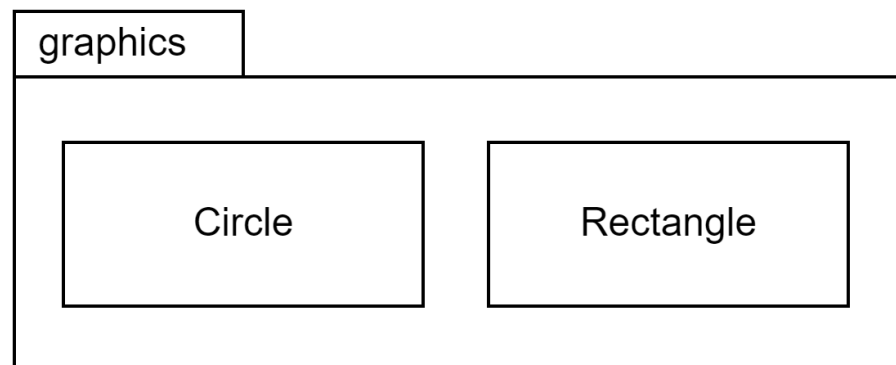
Anlegen von Paketen

- In der ersten Zeile wird festgelegt, zu welchem Paket eine Java-Klasse gehört

```
package graphics;  
  
public class Circle { /* ... */ }
```



```
package graphics;  
  
public class Rectangle { /* ... */ }
```





Anlegen von Paketen

- Wenn die `package`-Zeile fehlt, gehören die Klassen zu einem namenlosen Standardpaket (Default-Paket).
- Namenskonvention:
 - nur Kleinbuchstaben und Punkte
 - „reverse domain name pattern“ für Kollisionsfreiheit von Klassennamen: Der Domänenname wird in umgekehrter Reihenfolge dem Paketnamen vorangestellt; dann folgt der Name des umfassenden Pakets, dann Subpakete usw.
 - Beispiel `package de.ostfalia.prog.ss22.aufgabe1;`
- Die Eindeutigkeit der Paketnamen garantiert dann die Kollisionsfreiheit.
- Die Hierarchie der Paketnamen wird auf (Unter-)Verzeichnisse abgebildet (siehe später)



Pakete als Sichtbarkeitsgrenze

- Was in einem Paket deklariert ist, ist in anderen Paketen unsichtbar
- Zweck
 - In verschiedenen Paketen können gleiche Namen verwendet werden (Kollisionsfreiheit).
 - Programmierer müssen nicht Rücksicht nehmen, welche Namen schon woanders verwendet wurden.

```
package one;  
class C { /* ... */ }  
class D { /* ... */ }
```



```
package two;  
class D { // gleicher Name stört nicht (verschiedene Klassen)  
    C obj;      // Compilerfehler, C ist hier unsichtbar  
}
```





Programmieren

9. Modularisierung

9.2 Pakete: Import, Export



Export von Namen

- Namen können mit dem Zusatz `public` exportiert werden, sie sind dann in anderen Paketen sichtbar).
- Namen ohne Zusatz sind nur in diesem Paket sichtbar (default, Paketsichtbarkeit)
- `public`-Attribute und -Methoden werden nur exportiert, wenn die Klasse `public` ist
- Lokale Variablen und Parameter können nicht exportiert werden.

```
package one;
public class C {
    int x;
    public int y;
    void p() { /*... */ }
    public void q() { /*... */ }
    C() { /*... */ }
    public C (int x, int y) { /* ... */ }
}
```





Import von Typnamen

- Exportierte Klassennamen können in anderen Paketen importiert werden
 - durch gezielten Import der Klasse
 - durch Qualifikation mit dem Paketnamen

```
package mypack;  
  
import graphics.Circle;  
import one.C;  
  
public class MyClass {  
  
    Circle c;  
    C obj;  
  
}
```



```
package mypack;  
  
public class MyClass {  
  
    graphics.Circle c;  
    one.C obj;  
  
}
```





Import von Typnamen

- Exportierte Klassennamen können in anderen Paketen importiert werden
 - Durch Import aller `public`-Klassen eines Pakets
 - Nicht empfohlen!

```
package mypack;  
  
import graphics.*;  
import one.*;  
  
public class MyClass {  
  
    Circle c;  
    C obj;  
}
```





Import von Typnamen

- Beim Import müssen alle importierten Klassen verschiedene Namen haben

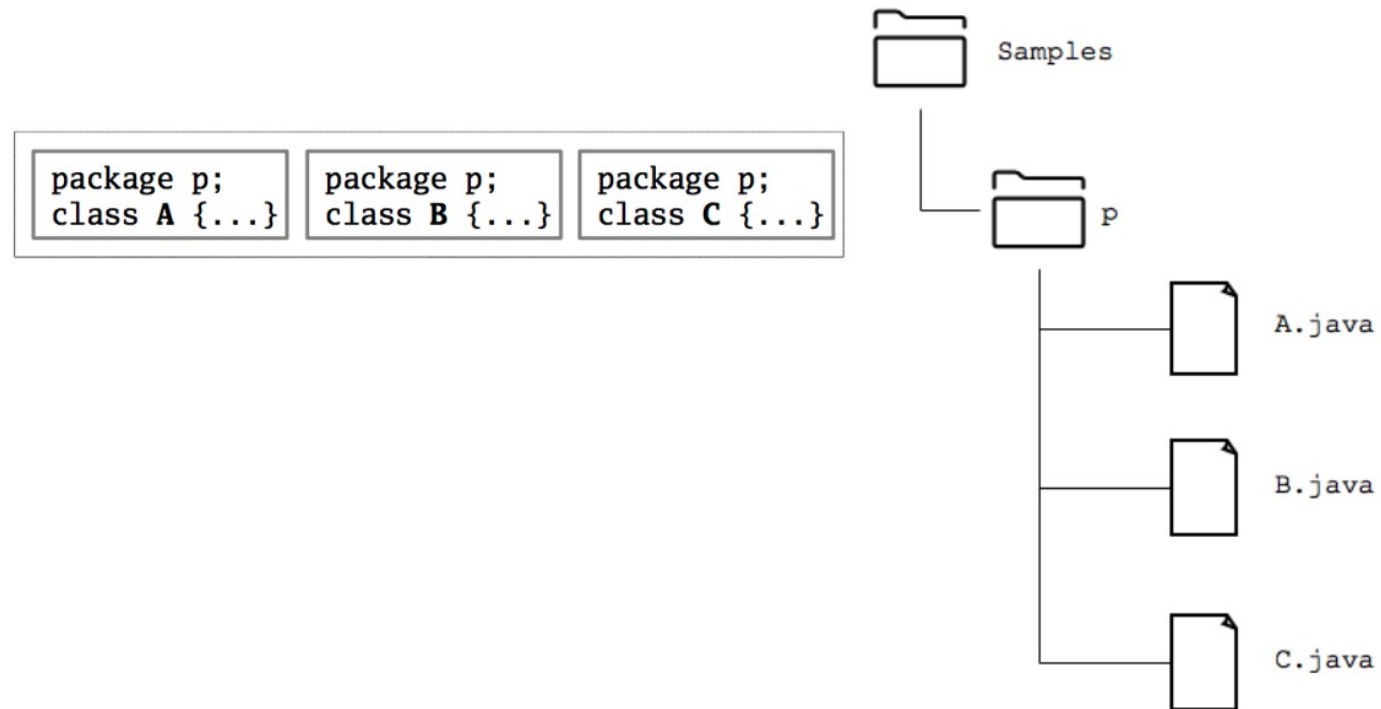
```
package mypack;  
  
import graphics.Circle;  
import javafx.scene.shape.Circle; //Compilerfehler: Kollision  
  
public class MyClass {  
    Circle c; // welche Klasse ist gemeint?  
}
```





Pakete und Verzeichnisse

- Pakete werden auf Verzeichnisse abgebildet, Klassen auf Dateien





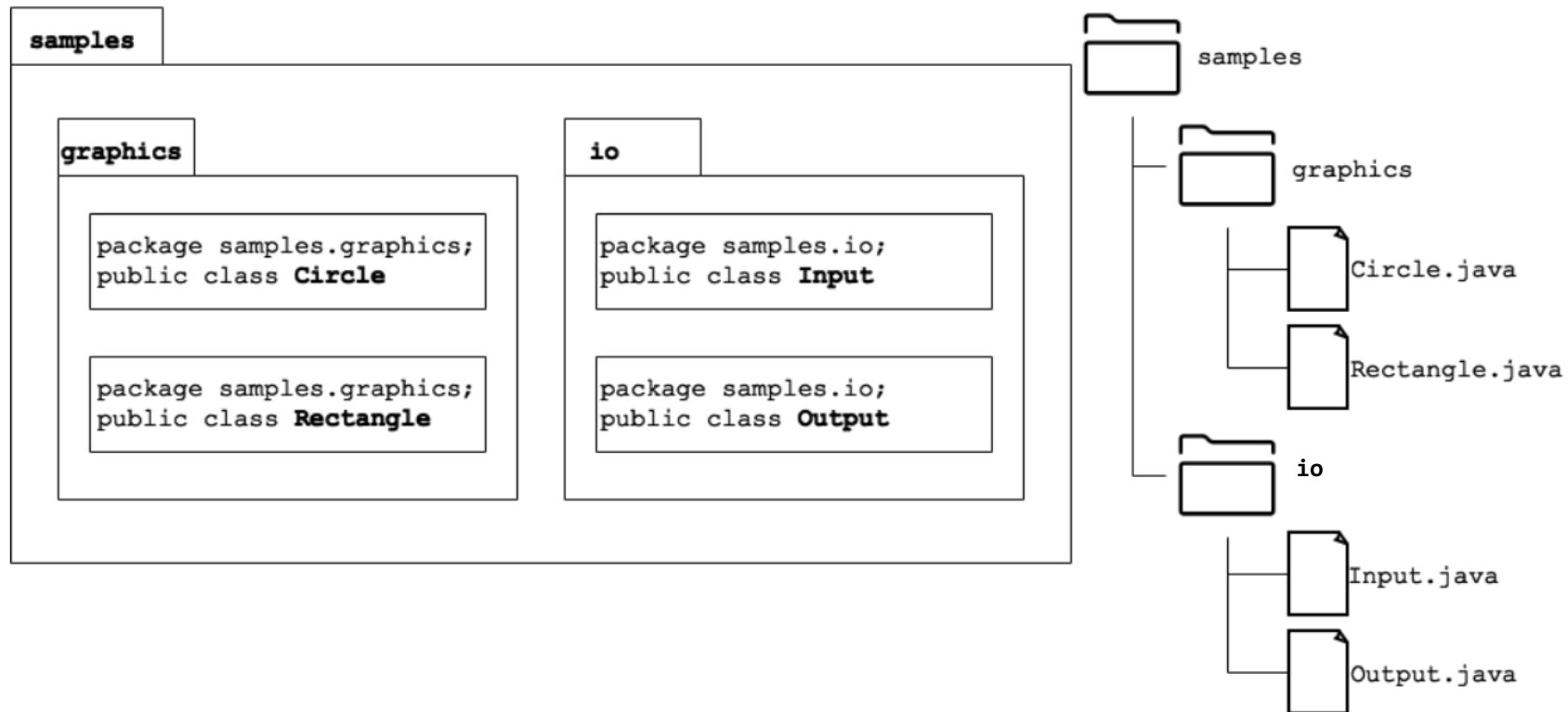
Pakete und Verzeichnisse

- Übersetzung und Ausführung mit dem JDK

```
code> javac p1/A.java  
code> java p1/A  
Hello World  
code> java p1.A  
Hello World  
code>
```

Geschachtelte Pakete

- Pakete können zu größeren Paketen zusammengefasst werden





Geschachtelte Pakete

- Import und Verwendung

```
package graphics;  
  
import graphics.a.b.c.D; // import der Klasse D  
import graphics.a.*;      // import aller public-Klassen aus graphics.a  
import graphics.*;        // import aller public-Klassen aus graphics  
                           // aber nicht aus den Subpaketen wie a, a.b, etc.  
  
public class A {  
    graphics.a.b.E myE;    // Qualifikation einer Klasse  
}
```





Geheimnisprinzip

- Information Hiding
- Prinzip
 - Verstecke die Implementierung komplexer Datenstrukturen vor Benutzern.
 - Erlaube den Zugriff auf die Daten nur über Methoden.
- Warum?
 - Verringert die Komplexität (Arbeiten mit den Daten wird einfacher).
 - Implementierung der Daten kann geändert werden, ohne dass Benutzer etwas merken.
 - Schützt vor mutwilliger oder unabsichtlicher Zerstörung.



Sichtbarkeitsattribute

	Example	UML	Class	Package	Subclass (same package)	Subclass (different package)	Everyone
private	private int a;	-	✓	✗	✗	✗	✗
<i>no modifier</i>	int b;	~	✓	✓	✓	✗	✗
protected	protected int c;	#	✓	✓	✓	✓	✗
public	public int d;	+	✓	✓	✓	✓	✓



Programmieren

9. Modularisierung

9.3 Innere Klassen



Innere Klassen

- Grundregel
 - Eine `public`-Klasse `C` ist in einer eigenen Datei `C.java` definiert.
- Ausnahme 1
 - Eine Klasse, die nicht `public` ist, darf auch in einer **anderen Datei** deklariert werden, nicht empfehlenswert.
- Ausnahme 2
 - Eine Klasse darf innerhalb einer anderen Klasse deklariert werden: **Innere Klasse**
- Ausnahme 3:
 - Eine innere Klasse muss keinen Klassennamen haben: **Anonyme innere Klasse**
 - Klasse wird dort definiert, wo sie (einmalig) benötigt wird.



Innere Klassen

- Motivation:
 - Erzeuge „Hilfsklassen“, die nur im Kontext einer gegebenen Klasse Verwendung finden, nahe an den Stellen, wo sie gebraucht werden
- Arten:
 - **Statische innere Klasse B**
 - Analog zu Klassenvariable in äußerer Klasse A definiert, Schlüsselwort `static`
 - Objekterzeugung: `A.B obj = new A.B(...);`
 - B hat Zugriff auf Klassenvariablen und Klassenmethoden von A
 - **Innere Instanzklasse B**
 - Analog zu Instanzvariablen in äußerer Klasse A definiert
 - Objekterzeugung nur im Verbund mit Instanz der umgebenden Klasse A
 - B hat Zugriff auf alle Variablen und Methoden von A
 - **Lokale innere Klasse B**
 - Analog zu lokaler Variablen in einer Methode, Konstruktor oder Block definiert
 - Zusätzlich Zugriff auf effektiv finale Parameter des umgebenden Blocks / Methode
 - **Anonyme innere Klasse**
 - Lokale Klasse, die ohne Klassenname in `new`-Anweisung definiert ist (Klassendefinition und Objekterzeugung sind in einer Anweisung zusammengefasst)
 - Zugriff wie bei lokaler innerer Klasse



Statische innere Klasse: Beispiel

- Deklaration, Objekterzeugung

```
public class Account {  
    private int userId;  
    private Permissions perm;  
  
    public Account(int userId) {  
        this.userId = userId;  
        perm = new Permissions();  
    }  
  
    public int getUserId() { return userId; }  
  
    public static class Permissions {  
        public boolean canRead;  
        public boolean canWrite;  
    }  
  
    public Permissions getPermissions() { return perm; }  
}
```

```
public static void main(String[] args) {  
    Account account = new Account(4711);  
    Account.Permissions p  
        = new Account.Permissions();  
    p = account.getPermissions();  
}
```



Innere Instanzklasse: Beispiel

```
public class Konto {  
  
    private int knummer;  
    private double saldo;  
    private Transaktion last;  
  
    public Konto(int k, double s) {  
        knummer = k; saldo = s;  
    }  
  
    public class Transaktion {  
  
        private String name;  
        private double betrag;  
  
        public Transaktion(String n, double b) {  
            name = n; betrag = b;  
        }  
  
        public String toString() {  
            return knummer + " " + name + " "  
                + betrag + " " + saldo;  
        }  
    }  
}
```



```
    public Transaktion getLast() { return last; }  
  
    public void zahleEin(double betrag) {  
        saldo += betrag;  
        last = new Transaktion("Einzahlung", betrag);  
    }  
  
    public void zahleAus(double betrag) {  
        saldo -= betrag;  
        last = new Transaktion("Auszahlung", betrag);  
    }  
  
    public static void main(String[] args) {  
        Konto k = new Konto(4711, 1000.0);  
        k.zahleEin(500.0);  
        k.zahleAus(700.0);  
  
        Konto.Transaktion t1 = k.new  
            Transaktion("Einzahlung", 100.0);  
  
        Konto.Transaktion t2 = k.getLast();  
        System.out.println(t1.toString());  
        System.out.println(t2.toString());  
    }  
}
```





Lokale innere Klasse: Beispiel

```
public class Liste {  
    private Element start, ende;  
  
    private static class Element {  
        private Object data;  
        private Element next;  
    }  
  
    public void add(Object o) {  
        if (o == null) return;  
        Element neu = new Element();  
        neu.data = o;  
        if (start == null)  
            start = ende = neu;  
        else {  
            ende.next = neu; ende = neu;  
        }  
    }  
}
```



```
public Iterator iterator() {  
  
    class IteratorImpl implements Iterator {  
        private Element e = start;  
  
        public boolean hasNext() { return e != null; }  
  
        public Object next() {  
            if (e == null) return null;  
            Object o = e.data; e = e.next;  
            return o;  
        }  
    }  
    return new IteratorImpl();  
}  
  
public static void main (String[] args) {  
    Liste liste = new Liste();  
    liste.add("Element 1"); liste.add("Element 2");  
    Iterator it = liste.iterator();  
    while (it.hasNext()) {  
        System.out.println(it.next());  
    }  
}
```





Anonyme innere Klasse: Beispiel

- Eine anonyme innere Klasse erweitert eine andere Klasse oder implementiert ein Interface, hat aber keine eigenen Konstruktoren
- Der Rückgabewert ist vom Typ der Klasse bzw. des Interface
- „Klassisches Beispiel“: Methode iterator()

```
public Iterator iterator2() {  
    return new Iterator() {  
        private Element e = start;  
  
        public boolean hasNext() {  
            return e != null;  
        }  
  
        public Object next() {  
            if (e == null) { return null; }  
            Object o = e.data;  
            e = e.next;  
            return o;  
        }  
    };  
}
```



Innere Klassen: weiteres Beispiel

```
public class CC {  
    int xC = 0;  
  
    int method9() {  
        DD meinD = new DD();  
        EE meinE = new EE();  
        return 9 + xC;  
    }  
  
    class DD {  
        int method10() {  
            return 10 + xC; // Zugriff auf Attribute der  
                           // umschließenden Klasse möglich  
        }  
    }  
  
    static class EE { // static: Zugriff nur auf static Elemente  
                     // der umschließenden Klasse, nicht auf  
                     // Instanzelemente  
        int method11() {  
            return 11; // nicht +xC;  
        }  
    }  
}
```

```
public class CCTest {  
  
    public static void main(String[] args) {  
        CC meinC = new CC();  
        int neun = meinC.method9();  
  
        // innere Instanzklasse (mit Referenz zum Objekt)  
        CC.DD meinD = meinC.new DD(); // Erzeugung  
        // CC.DD qualifizierter Klassenname  
        int zehn = meinD.method10();  
  
        // statische innere Klasse  
        CC.EE meinE = new CC.EE(); // Erzeugung  
        int elf = meinE.method11();  
    }  
}
```



Innere Klasse implementiert Interface: Beispiel

```
public class YnutztA {  
    InterA einA;  
  
    private class MeinA implements InterA {  
        @Override  
        public int method1() { return 1; }  
  
        // keine Schnittstellen-Methode  
        public int method6() { return 6; }  
    }  
  
    public YnutztA() {  
        einA = new MeinA();  
    }  
  
    public void ausgabe() {  
        System.out.println(einA.method1());  
        // System.out.println(einA.method6());  
        // nicht zugreifbar, da einA den  
        // statischen Typ InterA hat  
    } // und dort method6 unbekannt ist  
}
```

```
public interface InterA {  
  
    public int method1();  
}
```



Anonyme innere Klasse implementiert Interface

```
public class ZnutztAanonym {  
    InterA einA;  
    int welcherWert;  
  
    public ZnutztAanonym(int wert) {  
        this.welcherWert = wert;  
        einA = new InterA() {  
            @Override  
            public int method1() {  
                return welcherWert;  
            }  
        };  
    }  
  
    public void ausgabe() {  
        System.out.println(einA.method1());  
    }  
}
```





Anonyme innere Klasse implementiert Interface

- Eine anonyme innere Klasse wird üblicherweise verwendet, um eine Klasse „schnell“ zu erstellen, die ein Interface implementiert und nur an einer Stelle verwendet wird.
- Dies kann sogar in einem Methodenaufruf geschehen
- Parameter wird definiert, implementiert und erzeugt in „einem Schritt“

```
public void setA(InterA a) {  
    einA = a;  
}  
  
public static void main(String[] args) {  
    ZnutztAanonym z = new ZnutztAanonym(0);  
    z.setA(new InterA() {  
        @Override  
        public int method1() {  
            return 42;  
        }  
    });  
}
```



Programmieren

9. Modularisierung

9.4 Module



Java Module

- Java 9 führt neue Abstraktionsebene ein: Module
- Java Platform Module System (JPMS) oder Jigsaw
- Ziele:
 - Kapselung, feinere Zugriffskontrolle auch bei öffentlichen Klassen
 - Abstraktion, nur ein Teil der Klassen des Moduls wird auch als Schnittstelle angeboten
 - Explizite Abhängigkeiten zwischen Modulen



Sichtbarkeiten

- Klassen, Pakete und Module lassen sich als Container mit unterschiedlichen Sichtbarkeiten betrachten:
 - Ein Typ (Klasse oder Schnittstelle) enthält Attribute und Methoden.
 - Ein Paket enthält Typen.
 - Ein Modul enthält Pakete.
 - Private Eigenschaften in einem Typ sind nicht in anderen Typen sichtbar.
 - Nicht öffentliche Typen sind in anderen Paketen nicht sichtbar.
 - Nicht exportierte Pakete sind außerhalb eines Moduls nicht sichtbar.



Module

- Modul ist definiert
 - durch einen Namen
 - durch die Angabe, was es exportiert und
 - durch die Angabe, welches Modul es zur Arbeit selbst benötigt.
- Nicht alle öffentlichen Typen sind standardmäßig öffentlich
- Was tatsächlich öffentlich sichtbar ist, bestimmt das Modul
 - Typen, die das Modul *für alle* exportiert
 - Typen *für explizit aufgezählte* Module
 - alle Typen im gleichen Modul



Module

- Modultypen
 - Benannte Module
 - Systemmodule oder Plattformmodule
 - ...\\java\\jdk-17.0.2\\jmods
 - Anwendungsmodule
 - Automatische Module
 - Unbenanntes Modul

```
code> java --list-modules
java.base@17.0.2
java.compiler@17.0.2
java.datatransfer@17.0.2
java.desktop@17.0.2
java.instrument@17.0.2
java.logging@17.0.2
java.management@17.0.2
java.management.rmi@17.0.2
java.naming@17.0.2
...
jdk.unsupported.desktop@17.0.2
jdk.xml.dom@17.0.2
jdk.zipfs@17.0.2
```



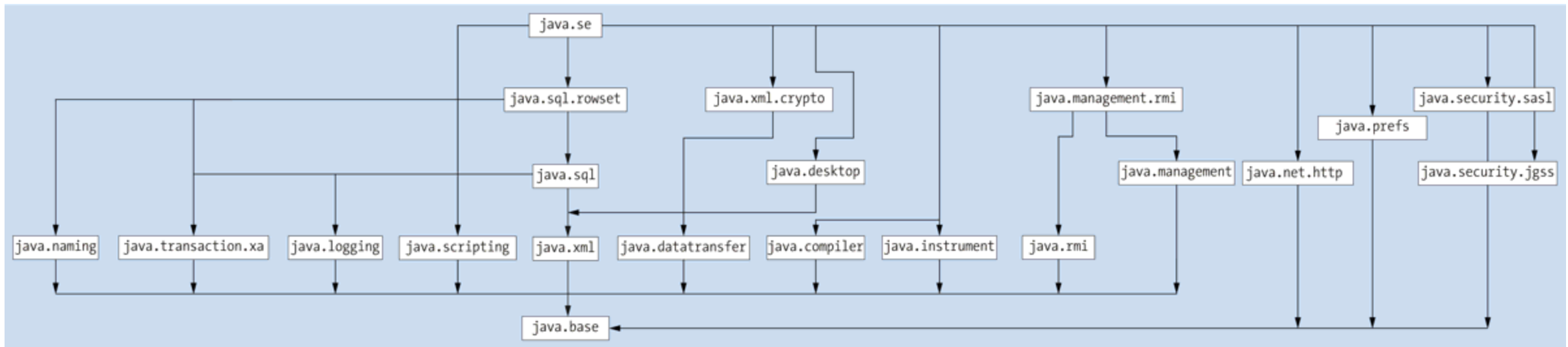
Module

- Werkzeug jmod zeigt unter anderem Information
 - Welchen Namen das Paket hat
 - Welche Pakete ein Modul exportiert (exports)
 - Welche Module benötigt werden (requires)
 - Plattform-Information

```
code> jmod describe ...\java\jdk-17.0.2\jmods\java.sql.jmod'  
java.sql@17.0.2  
exports java.sql  
exports javax.sql  
requires java.base mandated  
requires java.logging transitive  
requires java.transaction.xa transitive  
requires java.xml transitive  
platform windows-amd64
```

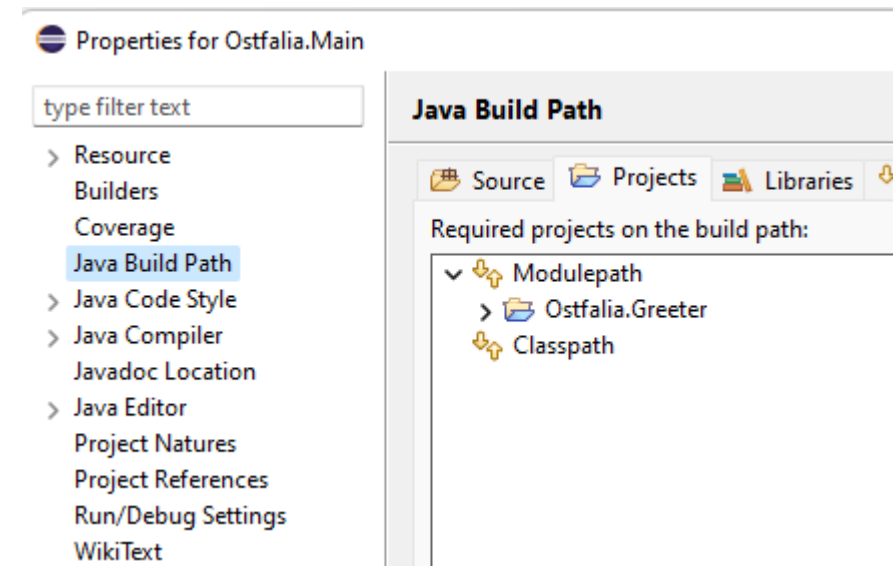
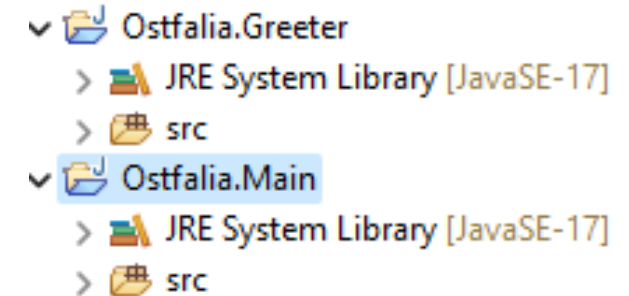


Modulübersicht Java SE



Beispiel in Eclipse

- Erstelle zwei Java Projekte (ohne Module)
- Projekt `Main` soll Projekt `Greeter` verwenden, wir benötigen dazu eine Abhängigkeit zwischen den Eclipse-Projekten





Beispiel in Eclipse

- Wir erstellen jeweils eine Klasse in einem Projekt, in Main wird die Klasse Greeter aus Ostfalia.Greeter verwendet

```
package de.ostfalia.greeter;

public class Greeter {
    private Greeter() { }

    public static Greeter instance() {
        return new Greeter();
    }
    public void greet(String name) {
        System.out.println("Hey " + name);
    }
}
```



```
package de.ostfalia.main;

import de.ostfalia.greeter.Greeter;

public class Main {

    public static void main(String[] args) {
        Greeter.instance().greet("Hallo");
    }
}
```

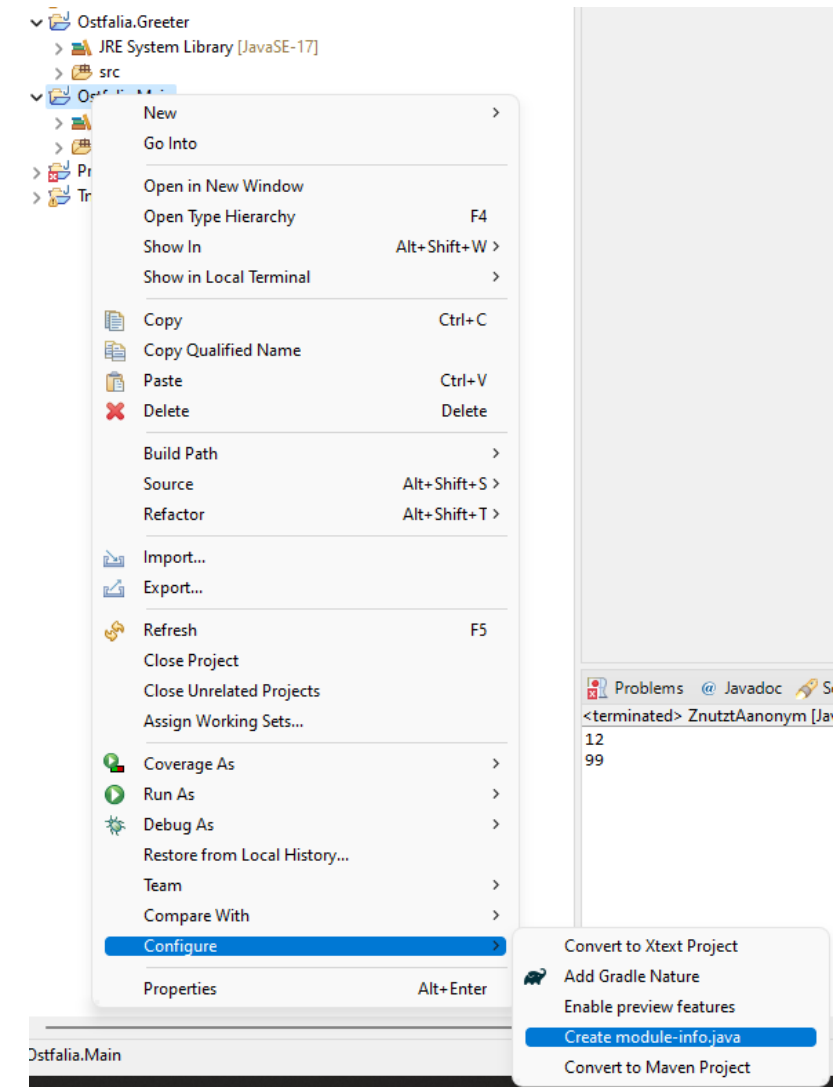


Beispiel in Eclipse

- Jetzt werden benannte Module erstellt

```
module Ostfalia.Greeter {  
    exports de.ostfalia.greeter;  
    requires java.base;  
}
```

```
module Ostfalia.Main {  
    exports de.ostfalia.main;  
    requires Ostfalia.Greeter;  
    requires java.base;  
}
```





Beispiel in Eclipse

- Einige Experimente mit den `module-info.java` Dateien

Modul	Aktion	Ergebnis
Ostfalia.Greeter Ostfalia.Main	<code>// requires java.base;</code>	Auskommentieren führt zu keiner Änderung, da <code>java.base</code> immer benötigt (required) wird.
Ostfalia.Greeter	<code>// exports de.ostfalia.greeter;</code>	Compilerfehler im main-Modul: »The type <code>de.ostfalia.Greeter</code> is not accessible«
Ostfalia.Greeter	<code>exports de.ostfalia.greeter to god;</code>	Nur das Modul <code>god</code> bekommt Zugriff auf <code>de.ostfalia.greeter</code> . Das main-Modul meldet »The type <code>de.ostfalia..greeter.Greeter</code> is not accessible«.
Ostfalia.Greeter	<code>exports de.ostfalia.closer;</code>	Das Hinzufügen führt zum Compilerfehler »The package <code>de.ostfalia.closer</code> does not exist or is empty«.
Ostfalia.Main	<code>// requires de.ostfalia.greeter;</code>	Compilerfehler »The import <code>de.ostfalia.greeter</code> cannot be resolved«
Ostfalia.Main	<code>// exports de.ostfalia.main;</code>	Kein Ergebnis, denn <code>c.t.i.m</code> wird von keinem Modul benötigt (required).



Beispiel in Eclipse

- Übersetzen und Packen von der Kommandozeile

```
eclipse-workspace> cd .\Ostfalia.Greeter\  
Ostfalia.Greeter> mkdir lib  
Ostfalia.Greeter> javac -d .\bin\ .\src\module-info.java .\src\de\ostfalia\greeter\Greeter.java  
Ostfalia.Greeter> jar --create --file=lib/Ostfalia.Greeter@1.0.jar --module-version=1.0 -C bin .  
Ostfalia.Greeter> jar --describe-module --file=lib/Ostfalia.Greeter@1.0.jar  
Ostfalia.Greeter@1.0 jar:file:/.../eclipse-  
workspace/Ostfalia.Greeter/lib/Ostfalia.Greeter@1.0.jar!/module-info.class  
exports de.ostfalia.greeter  
requires java.base
```

- Anlage des lib Ordners
- Übersetzen der Java-Datei in den bin-Ordner
- Erzeugung einer JAR-Datei (-C wechselt in bin und sucht dort nach Klassen)
- describe-module zeigt Modulinformation



Beispiel in Eclipse

- Übersetzen und Packen von der Kommandozeile

```
eclipse-workspace> cd .\Ostfalia.Main\  
Ostfalia.Main> mkdir lib  
Ostfalia.Main> javac -d bin --module-path ..\Ostfalia.Greeter\lib\ .\src\module-info.java  
                                                         .\src\de\ostfalia\main\Main.java  
Ostfalia.Main> jar -c --file=lib/Ostfalia.Main@1.0.jar --main-class=de.ostfalia.main.Main  
                                                         --module-version=1.0 -C bin .  
Ostfalia.Main> java -p 'lib;..\Ostfalia.Greeter\lib' -m Ostfalia.Main  
Hey Hallo
```

- Löschen/ Neuanlage des lib Ordners
- Übersetzen der Java-Datei in den bin-Ordner unter Angabe des Modulepfades
- Erzeugung einer JAR-Datei unter Angabe des Einstiegspunkts Main
- Start des Programms über das initiale Modul (-m)



Weitere Modultypen

- Automatische Module
 - Automatische Module entstehen, wenn (alte) JAR Dateien ohne Modulinformation in den Modulpfad aufgenommen werden (Kompatibilität)
 - Bekommt einen Namen vom System zugewiesen
 - Exportieren immer alle Pakete
- Unbenanntes Modul (eins)
 - Wenn eine JAR Datei in den `classpath` aufgenommen wird (aber nicht in den Modulpfad), werden die Klassen dem unbenannten Modul hinzugefügt (ebenfalls Kompatibilität)